Review

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 7

DEPENDENCY VS COUPLING

Dependency vs Coupling

Dependency

Occurs when one software modules **uses** another to function

A **uses** B

Dependencies aren't bad. They're necessary for modules in system to communicate

Examples:

- Function calling other functions
- Class calling methods in another class
- Code calls something in a library
- etc

Dependency vs Coupling

Coupling refers to the **degree of interdependence** between software modules

How strongly does A **use** B?

If two classes **depend strongly** on many details of each other, we say they are tightly coupled

We aim for loose coupling between modules

A class diagram provides (limited) hints at the degree of coupling

The strength of the connection between modules

Tight vs Loose coupling

Tight coupling is bad.

- Two modules **strongly depend** on each other.
- Changing one **requires changing** the other.

Loose Coupling is good.

- Two modules **weakly depend** on each other.
- Changing one **requires little or no change** to the other

Loose coupling

We **aim** for **loose coupling**

Loose coupling makes it possible to:

- **Understand** one class without reading others
- **Change** one class with little or no effect on other classes.

Loose coupling increases maintainability

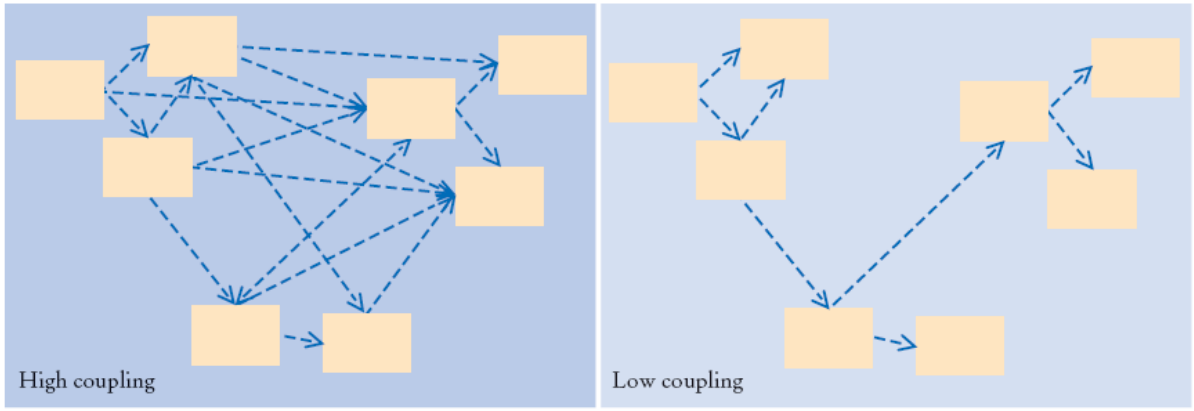


Figure 2 High and Low Coupling Between Classes

Tight coupling is bad

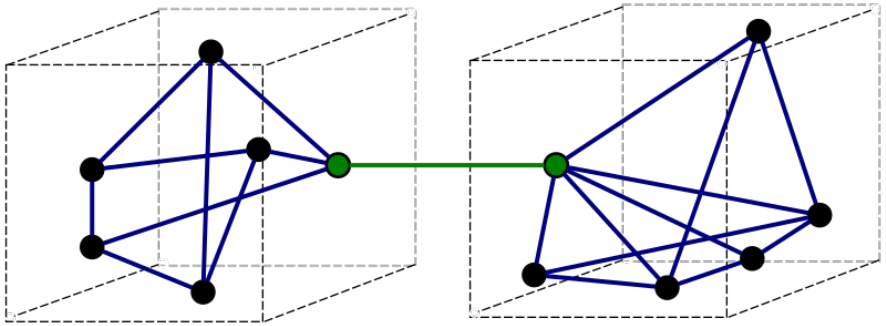
We want to **avoid** **tight coupling**

Changes to one class bring a **cascade of changes** to other classes

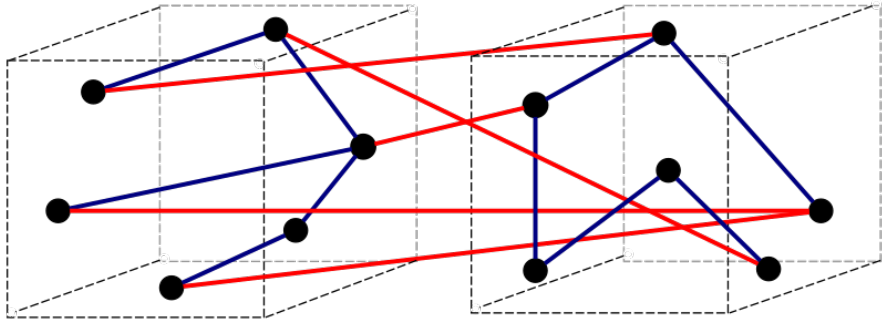
Classes are **harder to understand** in isolation

Flow of control between objects of different classes is **complex**

Difficult to test classes in isolation



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

Tight coupling

How to identify **Tight coupling:**

1. Code depends on **concrete implementations**
2. Code has **knowledge of other Class' internal data attributes**
3. **Hard to use different implementation** without refactoring
4. Hard to test code in isolation
5. Changes in code **affect other areas**

Loose coupling

How to identify **Loose coupling:**

1. Code **depends on abstractions** (abstract classes and interfaces)
2. Code only has **knowledge of other Class' public methods**
3. **Easy to use different implementation** without refactoring
4. Easy to test code in isolation
5. Changes in code **don't affect other areas**

coupling_examples.py
coupling_two_way.py

Cohesion

Cohesion refers to the number and diversity of **tasks** that a **single unit** is responsible for

If each unit is responsible for one single logical task, we say it has high cohesion

We aim for high cohesion inside modules

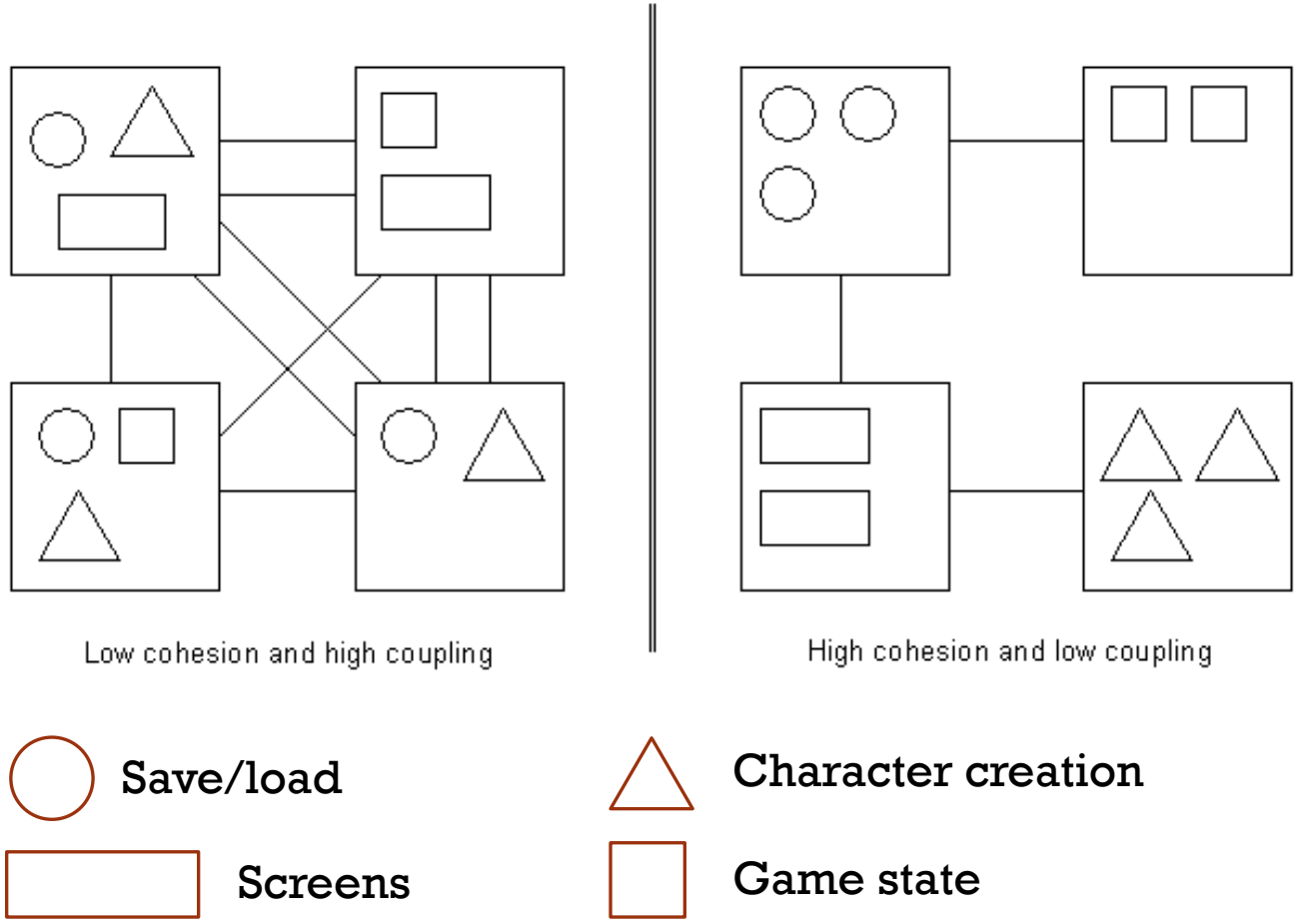
'Unit' applies to functions, classes, and modules

The glue that holds a module together

High cohesion

High cohesion makes it easier to:

1. Understand what a class or method does
2. Use descriptive and accurate names for variables, methods, and classes
3. Reuse classes and methods (we love this!)



High cohesion examples

Class level:

- Classes should represent one single, well-defined **entity**

Method level:

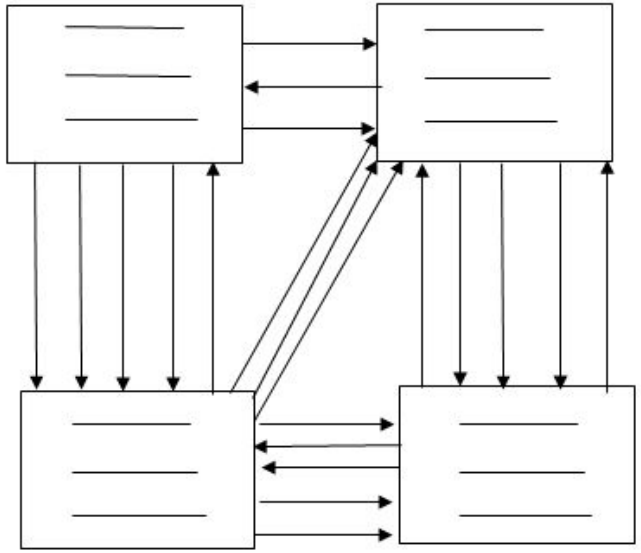
- A method should be responsible for one and only one well defined **task**

We **avoid** loosely cohesive classes and methods

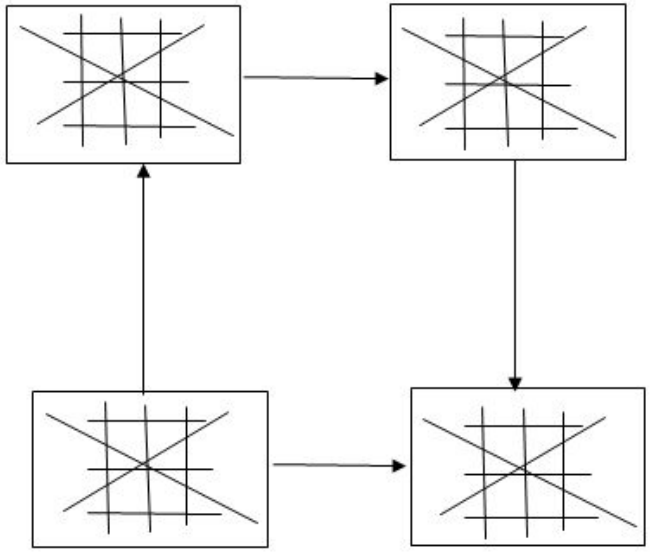
- Methods perform multiple tasks # BAD
- Classes have no clear identity # BAD

Design Principle – Coupling and Cohesion

Examples of Coupling and Cohesion



High Coupling
Low Cohesion



Low Coupling
High Cohesion

Which one is better from a software design point of view and why?

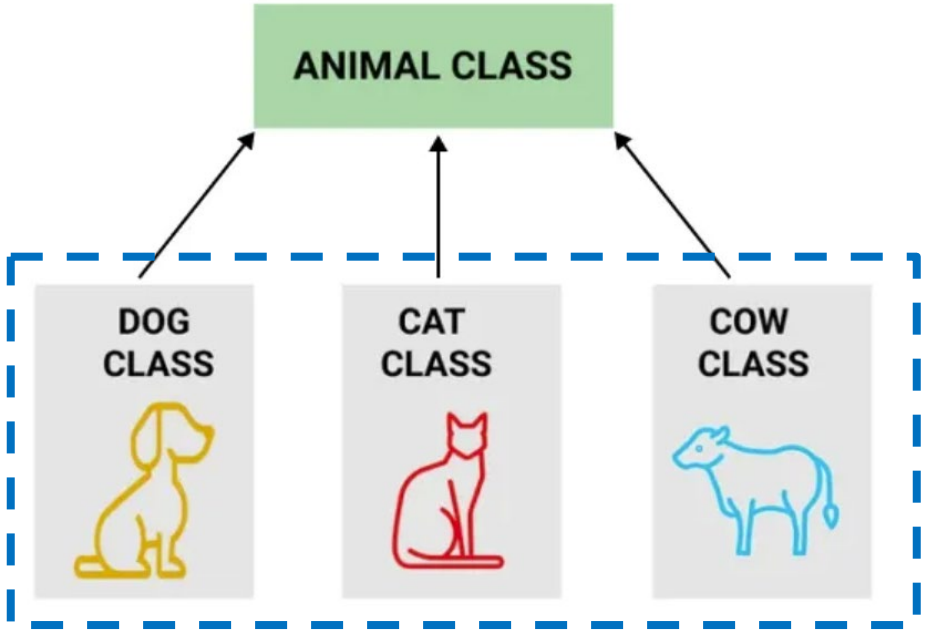
coupling_low_cohesion.py
coupling_high_cohesion.py

INHERITANCE

Inheritance and polymorphism

Inheritance is a fundamental concept in object-oriented programming.

It allows a **class** (called a child or derived class) to **inherit attributes and methods** from **another class** (called a parent or base class)



https://www.geeksforgeeks.org/python/inheritance-in-python/

Why use inheritance?

- Promotes **code reusability** by **sharing attributes and methods** across classes
  - Dog class inherits **_name** and **speak methods** from Animal

```
class Animal:
    def __init__(self, name):
        self._name = name

    def speak(self):
        print("The animal makes a sound")
```

```
class Dog(Animal):
    pass
```

https://www.geeksforgeeks.org/python/inheritance-in-python/

Why use inheritance?

- Models real-world hierarchies like **Animal -> Dog** or **Person -> Employee**.
- **Simplifies maintenance** through **centralized updates** in parent classes.
  - Parent Animal class added new **eat method**, that Dog class will also inherit

```
class Animal:
    def __init__(self, name):
        self._name = name

    def speak(self):
        print("The animal makes a sound")

    def eat(self):
        print("The animal eats")
```

```
class Dog(Animal):
    pass
```

https://www.geeksforgeeks.org/python/inheritance-in-python/

Why use inheritance?

- Enables **method overriding** for **customized subclass behavior**.
  - Dog class **overrides speak** method with **new print statement**

```
class Animal:
    def __init__(self, name):
        self._name = name

    def speak(self):
        print("The animal makes a sound")

    def eat(self):
        print("The animal eats")
```

```
class Dog(Animal):
    def speak(self):
        print("Woof!")
```

https://www.geeksforgeeks.org/python/inheritance-in-python/

Why use inheritance?

- Supports scalable, extensible design using **polymorphism**.
  - Scalable: Can add as many **new Animal subclasses** as we want
  - Extensible: **New behaviors** can be added in subclasses without changing Animal base class
  - Polymorphism: The same **speak()** method performs **different behaviors** depending on the animal object called

```
class Animal:
    def __init__(self, name):
        self._name = name

    def speak(self):
        print("The animal makes a sound")
```

```
class Dog(Animal):
    def speak(self):
        print("Woof!")

    def fetch(self):
        print("Fetching ball")
```

```
class Cat(Animal):
    def speak(self):
        print("Meow!")
```

https://www.geeksforgeeks.org/python/inheritance-in-python/

Inheritance in Python

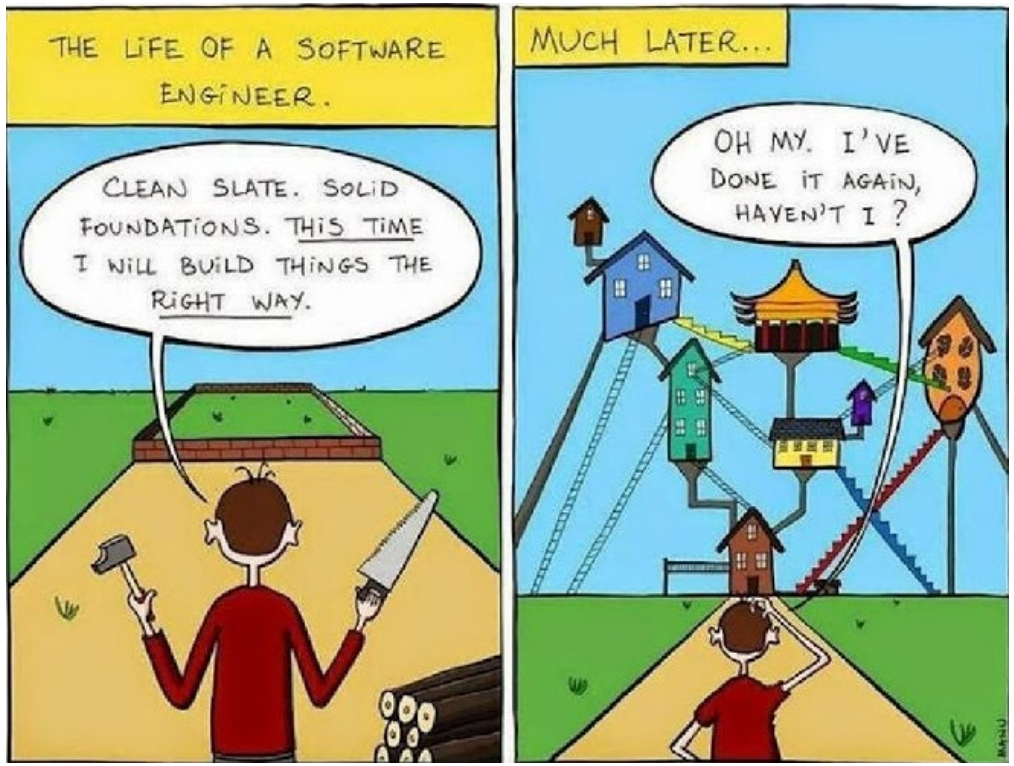
So easy

Almost the same approach as Java:

- Push common attributes as far up the inheritance hierarchy as possible
- Represent an **is-a** relationship
- **Subclass** (child class) is a specialized version of the **superclass** (parent class)

With one big difference:

- Multiple inheritance (oh the horror!)



Basic inheritance in Python

We specify the superclass as a parameter to the class definition:

```
class Subclass(Superclass):
    # class definition here
```

INTERFACE VS ABSTRACT CLASS

Interface vs abstract class

Both represent some abstract concept where we know the behaviors they can perform, but we don't know the exact implementation just yet.

Interfaces and **abstract classes** are ***incomplete***. They are NOT meant to be instantiated

Challenge: Write code to calculate the area of some **Shapes** like **Triangle** and **Circle**

Initial implementation: We should have a **Shape base class** and **incomplete area method**. Can't calculate the area until we have a specific shape

```
class Shape:
    def area(self):
        pass
```

Interface vs abstract class

Use **interfaces** in situations where there are **no attributes** and **only abstract methods**

Unimplemented methods will be defined in the child classes.

The code to the right is an interface. Notice there are no attributes, and the **incomplete area method**

```
import abc
class Shape(abc.ABC):

    @abc.abstractmethod
    def area(self):
        pass
```

Interface

- Inherits from **ABC**
- Has no attributes. Ie: no `self.my_var = var`
- Has only abstract methods
- Has **abstract method signatures**, **no implementation**

```
import abc
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self):
        pass
```

Child classes implement the Interface

- **Inherit from interface**
- **Complete overridden method implementations**

Can have:
- Extra **attributes** and methods

Classes containing **no abstract methods** also called Concrete classes

```
class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height


class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2
```

Interface vs abstract class

Challenge: Write code to represent two **Characters**, a **Troll** and **Player**. Both characters start with 100 health. There should be a way to access this health. Trolls can attack with 10 damage, Players attack with 50 damage

In this scenario, **health** is a common attribute amongst **Troll** and **Player**. The ability to get the health is also shared

The attack behavior is unknown in the Character class, but known in each specific character class (Troll and Player)

Initial implementation: We should have a **Character base class** to encapsulate the **health attribute** and **accessor**, and the **incomplete attack behavior**

```
class Character:
    def __init__(self):
        self.health = 100

    def get_health(self):
        return self.health

    def attack(self, enemy):
        pass
```

Interface vs abstract class

Use **abstract classes** in situations where there is at **least one abstract method.** Abstract classes may contain **attributes** and other **implemented methods.**

Unimplemented method will be defined in the child classes

The code to the right is an abstract class. Notice the **health attribute** and **accessor method**, and the **incomplete attack method**

```
import abc
class Character(abc.ABC):
    def __init__(self):
        self.health = 100

    def get_health(self):
        return self.health

    @abc.abstractmethod
    def attack(self, enemy):
        pass
```

Abstract class

- Inherits from **ABC**
- Has at least one **abstract method signature**
  - **may have implementation**
- Has at least one **attribute** or **fully implemented method**

Can have:
Extra attributes and other implemented methods

```
import abc
class Character(abc.ABC):
    def __init__(self):
        self.health = 100

    def get_health(self):
        return self.health

    @abc.abstractmethod
    def attack(self, enemy):
        pass
```

Child classes implement the Abstract class

- **Inherit from abstract class**
- **Complete overridden method implementations**

Can have:
- Extra attributes and methods
Classes containing **no abstract methods** also called Concrete classes

```
class Troll(Character):
    def attack(self, enemy):
        enemy.health -= 10
        print("Troll attacks!")

class Player(Character):
    def attack(self, enemy):
        enemy.health -= 50
        print("Player attacks!")
```

Most abstract

Most concrete

Interface

- Only abstract methods
- No implemented methods and attributes
- Can NOT be instantiated

Abstract Class

- At least one abstract method
- Can have implemented methods and attributes
- Can NOT be instantiated

Concrete Class

- No abstract methods
- Can have implemented methods and attributes
- CAN be instantiated

SOLID Design Principles

- A set of guidelines to help us write Maintainable, decoupled, flexible Object Oriented Code
- Created by Robert C. Martin, also known as Uncle Bob amongst developers.
- Highly respected and has a bunch of books on writing clean code.

Single Resposibility Principle
A class should have only a single responsibility (i.e. only one potential change in the software's specification should be able to affect the specification of the class)

Open / Closed Principle
A software module (it can be a class or method) should be open for extension but closed for modification.

Liskov Substitution Principle
Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.

Interface Segregation Principle
Clients should not be forced to depend upon the interfaces that they do not use.

Dependency Inversion Principle
Program to an interface, not to an implementation.

Single Responsibility Principle

"The Single Responsibility Principle requires that each class is responsible for only one thing."

Ask yourself, are there multiple reasons to change my class?

There should only be one reason to change your class. Each class should have one and only one responsibility.

Single Responsibility Principle

Think of the Transaction class in Assignment 1

Conceptually what is a transaction in the real world?

In physical form it's kind of like a receipt. It indicates

- Money spent
- Location
- Date/time

In the real world, does the receipt also have the ability to

- Query you for information?
- Ask you about your budget?
- Have access to your bank account?

Single Responsibility Principle

The **single responsibility** of the transaction class is to contain information about a transaction (money, location, date/time etc)

It should not have any other responsibility (UI interactions, access to other classes)

Other responsibilities should be handled by other classes

- UI Manager's single responsibility is to query user about different types of information and return that information to whatever class needs it
- User class' single responsibility is to contain all the information that makes up a user

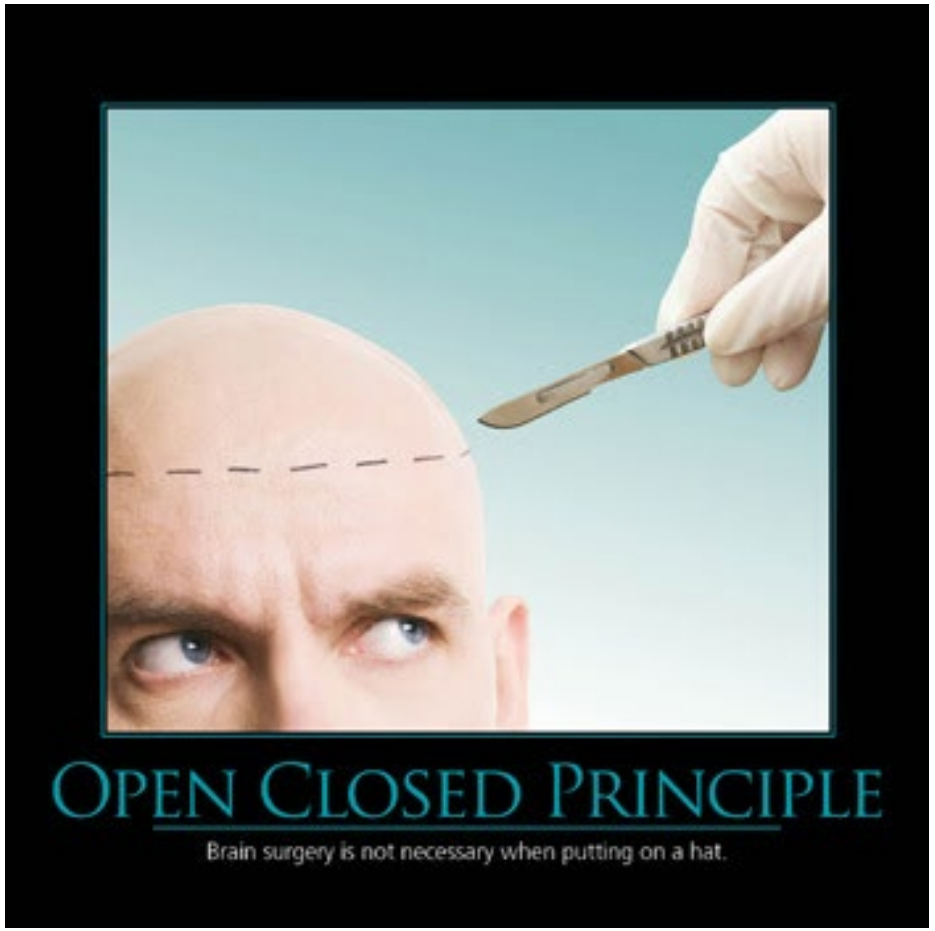
If one class becomes too bloated and is taking on too much responsibility, split it into multiple classes

Open Closed Principle

The Open/Closed Principle states that software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

Once a class is made, its behaviour shouldn't be modified to **support special cases**. It should be **CLOSED for modification**.

Instead, we extend (or inherit) from the class and override behaviours. That is, it's **OPEN for extension**.



How do we do this?

We accomplish this by:

1. Partitioning our software system into components
2. Arranging the components into a dependency hierarchy that protects higher-level components from changes in lower level components
3. Avoiding modifying source code in classes that are already in use
4. Using abstract base classes in inheritance hierarchies as interfaces

Our goal is (as always) to make a system easy to extend without incurrent a high impact of change

Software changes

We don't want to modify code that's in use

But sometimes we have to make changes for special cases

We **should extend the original object**:

- *Leave original source code untouched*.

Sounds like polymorphism doesn't it!

Challenges:

- We have to anticipate changes
- Designer must be aware of possible variations in current functionality, consider a technical interface that will scale.

Example

Imagine we're asked to write code to calculate area of a rectangle

```
class Rectangle:
    def __init__(self, w, h):
        self.width = w
        self.height = h
```

```
#area function somewhere in driver.py
def area(r):
    return r.width * r.height
```

Example

Asked to now find area of rectangle and circle

```
class Shape:
    pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.width = w
        self.height = h

class Circle(Shape):
    def __init__(self, r):
        self.radius = r
```

```
#area function somewhere in driver.py
def area(shape):
    if shape is Rectangle:
        return shape.width * shape.height
    else:
        return shape.radius * shape.radius * 3.14
```

Example

Asked to now find area of rectangle and circle, and triangle…

```
class Shape:
    pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.width = w
        self.height = h

class Circle(Shape):
    def __init__(self, r):
        self.radius = r

class Triangle(Shape):
    #triangle code
    pass
```

```
#area function somewhere in driver.py
def area(shape):
    if shape is Rectangle:
        return shape.width*shape.height
    elif shape is Circle:
        return shape.radius * shape.radius * 3.14
    else:
        #triangle area code
        pass
```

Example - Solution

Abstract out **Shape** and have **subtypes** with their own **area** functions

After creating the **Shape** interface and submitting it to repo, it's considered **closed** for modification. But it's always **open** to extend functionality through subclasses

```
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.width = w
        self.height = h
    def area(self):
        return self.width * self.height
```

```
class Circle(Shape):
    def __init__(self, r):
        self.radius = r
    def area(self):
        return self.radius * self.radius * 3.14


#area function somewhere in driver.py
def area(shape):
    return shape.area()
```

Liskov Substitution Principle

In programming, the Liskov substitution principle states that if ***S*** is a subtype of ***T***, then objects of type ***T*** may be replaced (or substituted) with objects of type ***S***.

Or

Objects of the base class must be replaceable by objects of its subclasses without breaking the program.

Liskov Substitution Principle

We can **substitute** passing in a Shape, Rectangle, or Circle object into the **area** function, and it should work

If for some reason calling shape.area() doesn't work with Shape or any of its subclasses, then the function fails the Liskov substitution principle

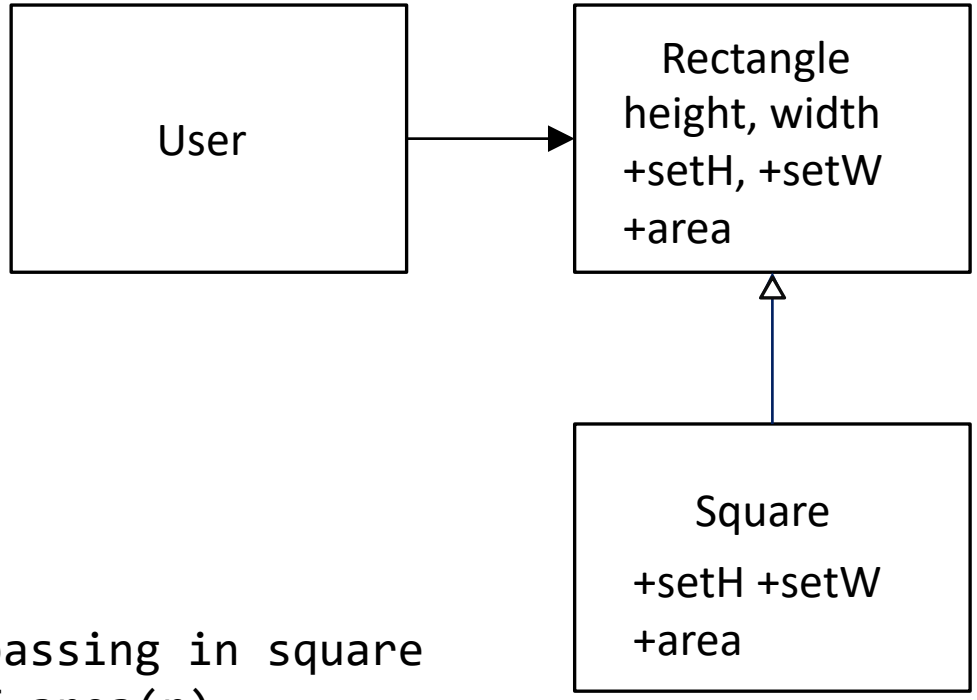
```
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.width = w
        self.height = h
    def area(self):
        return self.width * self.height
```

```
class Circle(Shape):
    def __init__(self, r):
        self.radius = r
    def area(self):
        return self.radius * self.radius * 3.14
```

```
def area(shape : Shape):
    return shape.area()
```

Failing Liskov Sustitution Principle




```
//passing in square
def area(r)
  r.setW(5) #sets height & width 5
  r.setH(2) #sets height & width 2
  assert(r.area() == 10) # Error area = 4
```

Fails Liskov Substitution Principle

Square is not a proper subtype of Rectangle

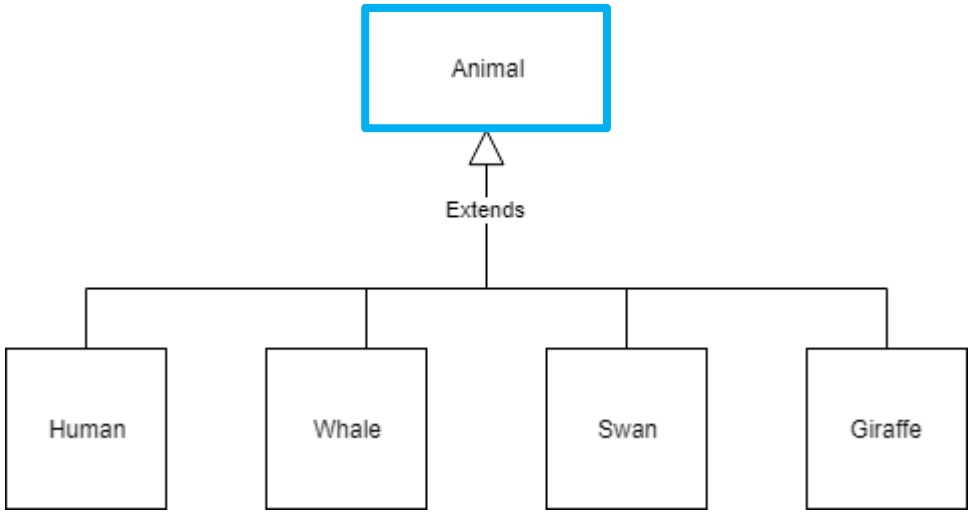
The height and width of rectangle are independently mutable

In contrast, the height and width of square must change together

The user will get confused because they will think they are dealing with a rectangle

liskov_fail.py

Interface Segregation Principle: Zoo Problem




```
class Animal(abc.ABC):

    def breathe(self):
        # common breathing code here

    def eat(self):
        # common eating code here

    @abc.abstractmethod
    def walk(self):
        # common walking code

    @abc.abstractmethod
    def swim(self):
        # common swimming code

    @abc.abstractmethod
     def fly(self):
        # common flying code
```

So we followed all our design principles and wrote the animal ABC Class.
But this is terrible! **Now our Human class needs to override the fly method. Our Whale can walk!**

Interface Segregation Principle

Many client-specific interfaces are better than one general-purpose interface.

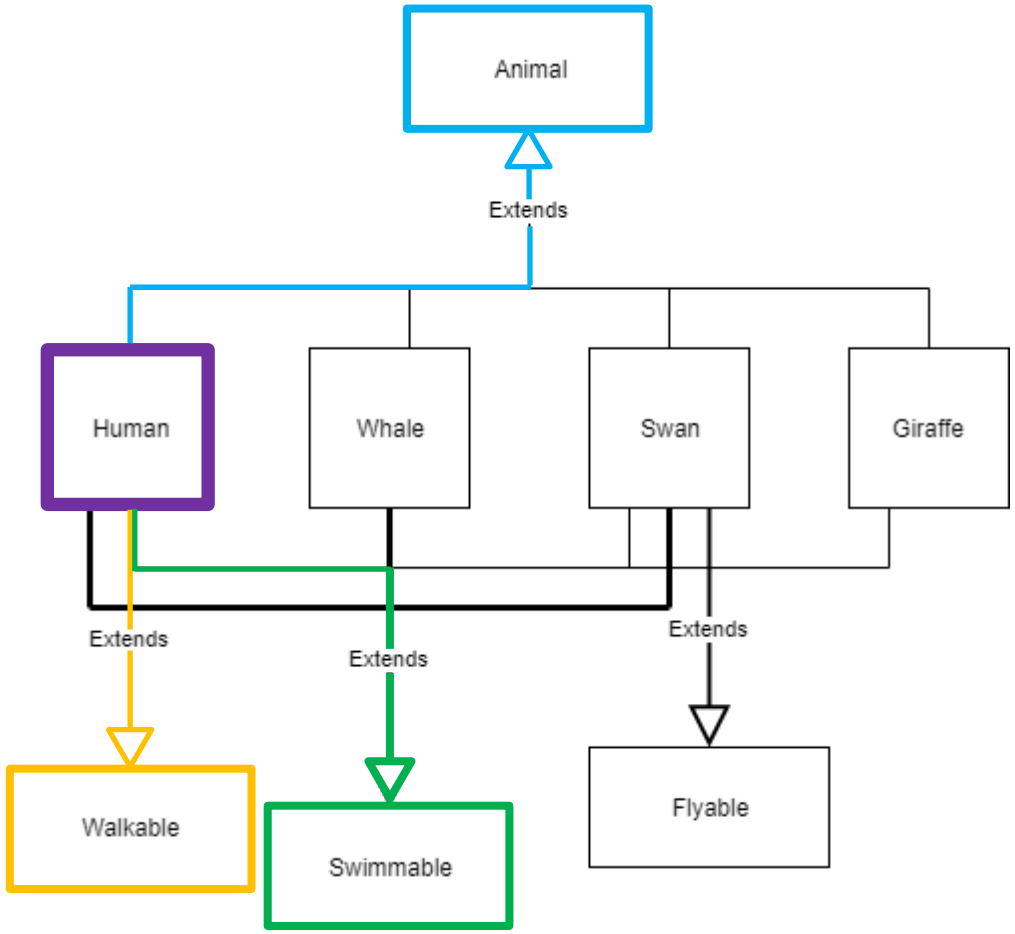
Or

The interface segregation principle states that no client should be forced to depend on methods it does not use.

Put more simply: Do not add additional functionality to an existing interface by adding new methods.

This is the Single Responsibility Principle for Interfaces/Abstractions!

Back To Our Zoo!




```
class Walkable(abc.ABC):
    @abc.abstractmethod
    def walk(self):
        pass
class Swimmable(abc.ABC):
    @abc.abstractmethod
    def swim(self):
        pass
class Human(Animal, Walkable, Swimmable):
    def walk(self):
        # overridden walking code
    def swim(self):
        # overridden swimming code
```

Instead of polluting the **Animal** base class with additional functionality, we create separate interfaces (read: ABC) that handle different responsibilities.

This may seem difficult to maintain, but in fact it isn't.

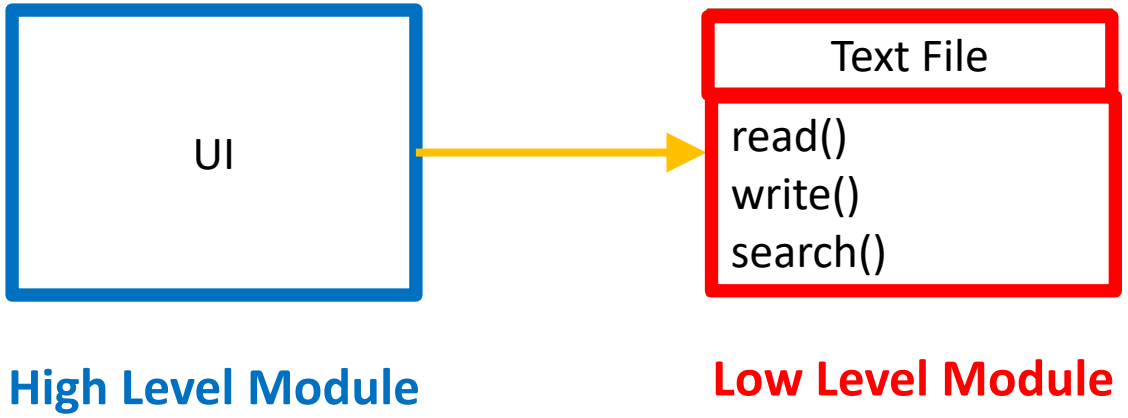
Dependency Inversion Principle

Dependency Inversion Principle can be thought as a combined effect of the previous principles.

Specifically, the Open Closed Principle and Liskov Substitution Principle

Before we dive into this, we need to look at a few concepts.

High Level & Low Level Modules

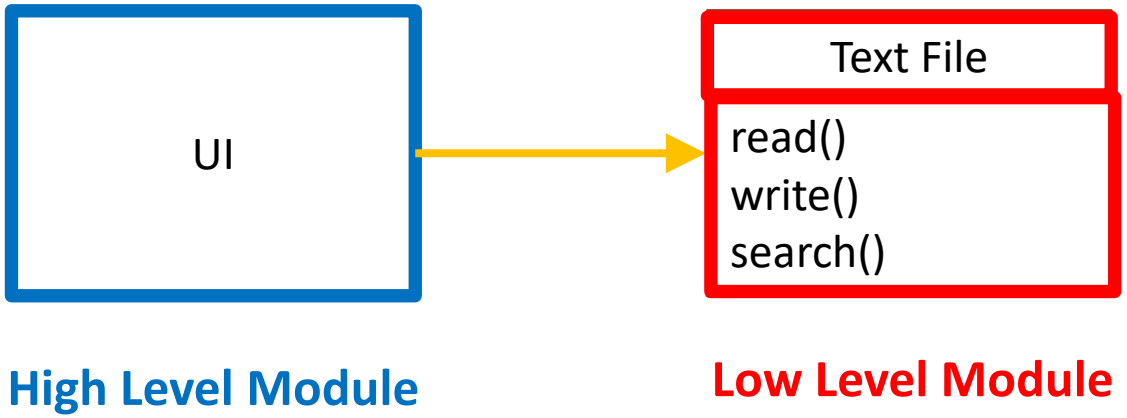


High Level Modules are the ones that contain complex logic made up of small 'chunks' of simpler code. For example, our UI class

A **Low Level Module** on the other hand, is a class that encapsulates some simple atomic behaviour. Such as writing and reading from a file, displaying an image, etc.

We generally compose high level modules with low level modules. That is, **high level modules are Coupled and Dependent on low level modules**

High Level & Low Level Modules

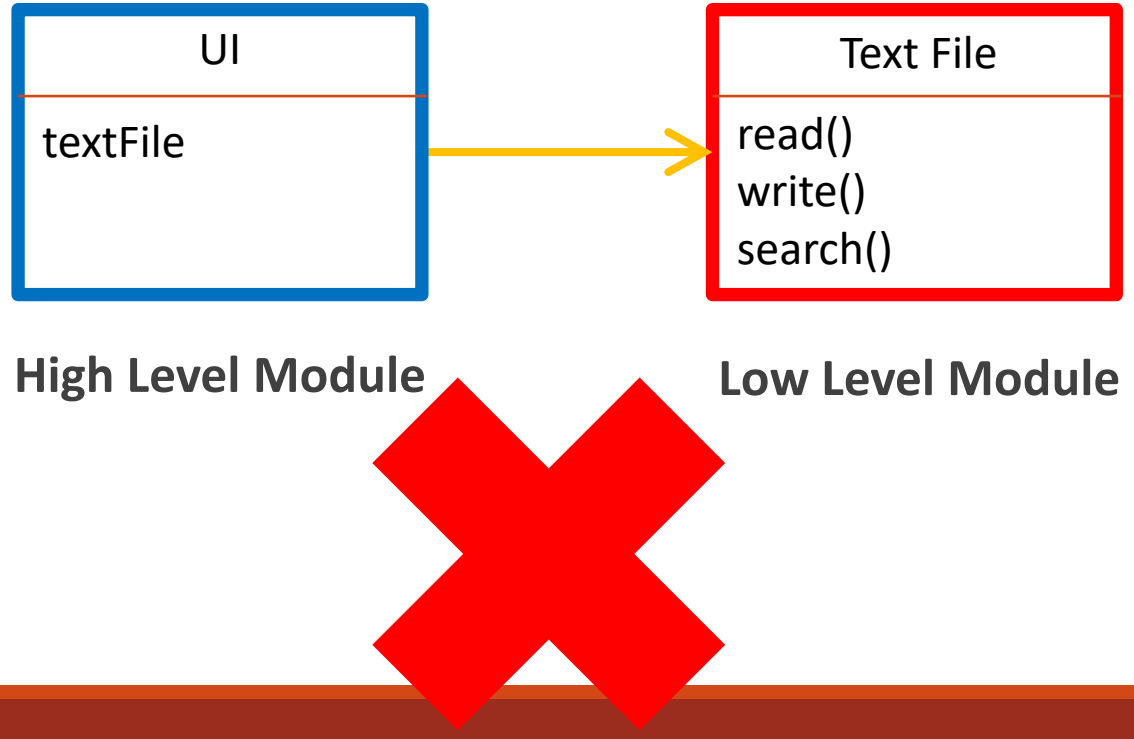


<u>This is not very good design.</u>

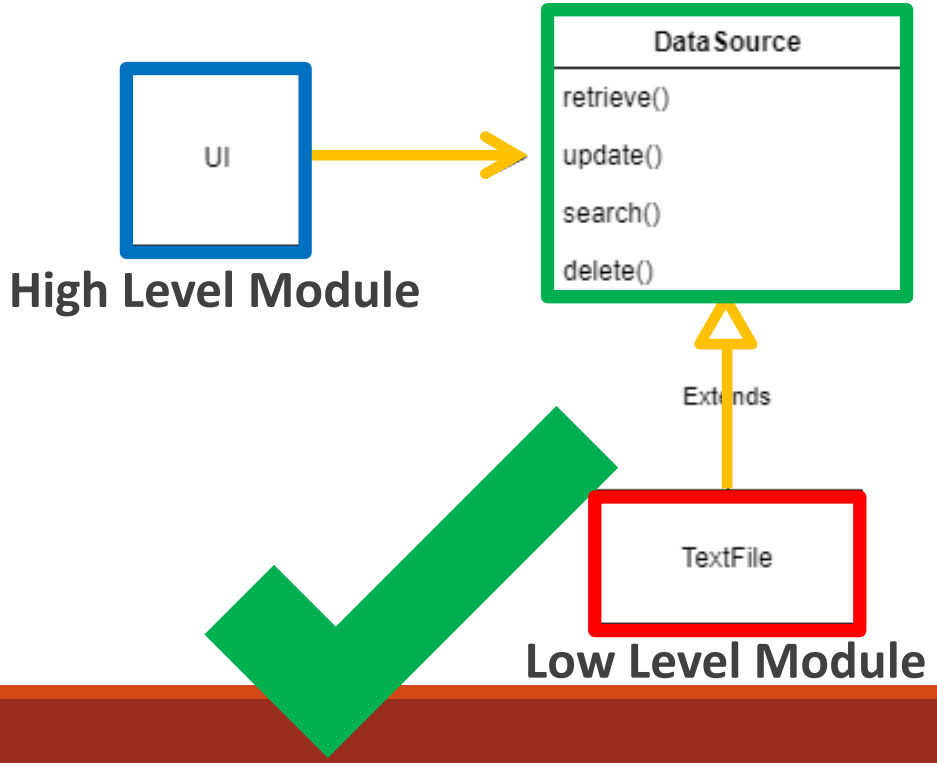
We don’t want this dependency. We want to be able to change or replace the lower level modules without tampering with the complex code in the higher level modules

Dependency Inversion Principle

1. **High-level modules** should not **depend** on **low-level modules**.

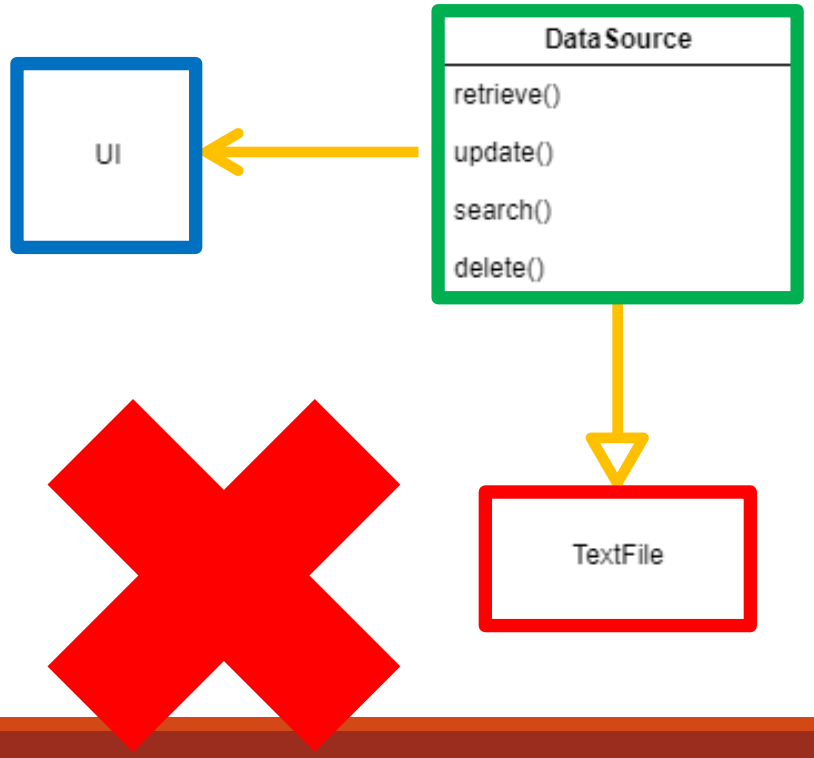


High-level modules and **low-level modules** should **depend** on **abstractions**.

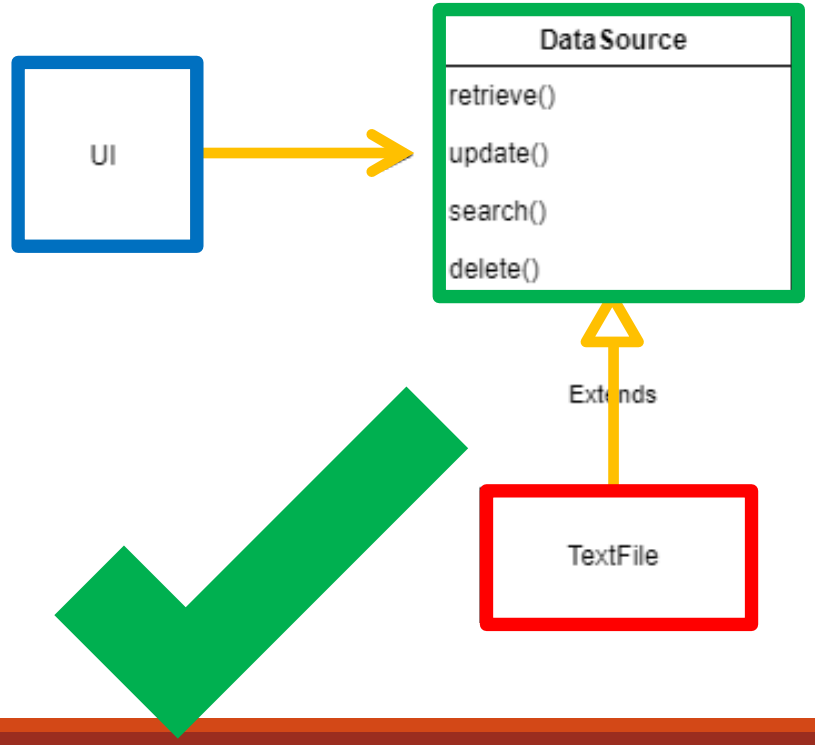


Dependency Inversion Principle

2. **Abstractions** should not **depend** on details (**UI**, **TextFile**).



Details (**UI**, **TextFile**) should **depend** on **abstractions**.

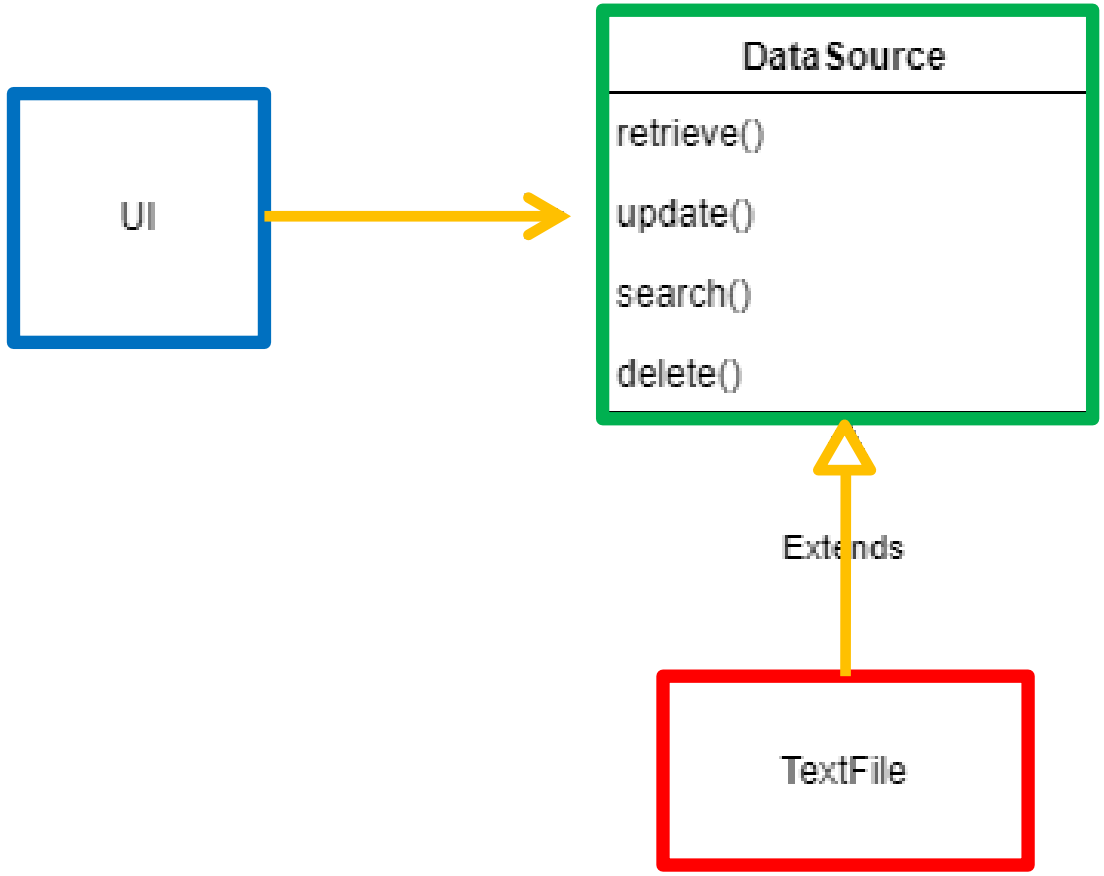


We inversed the dependencies!

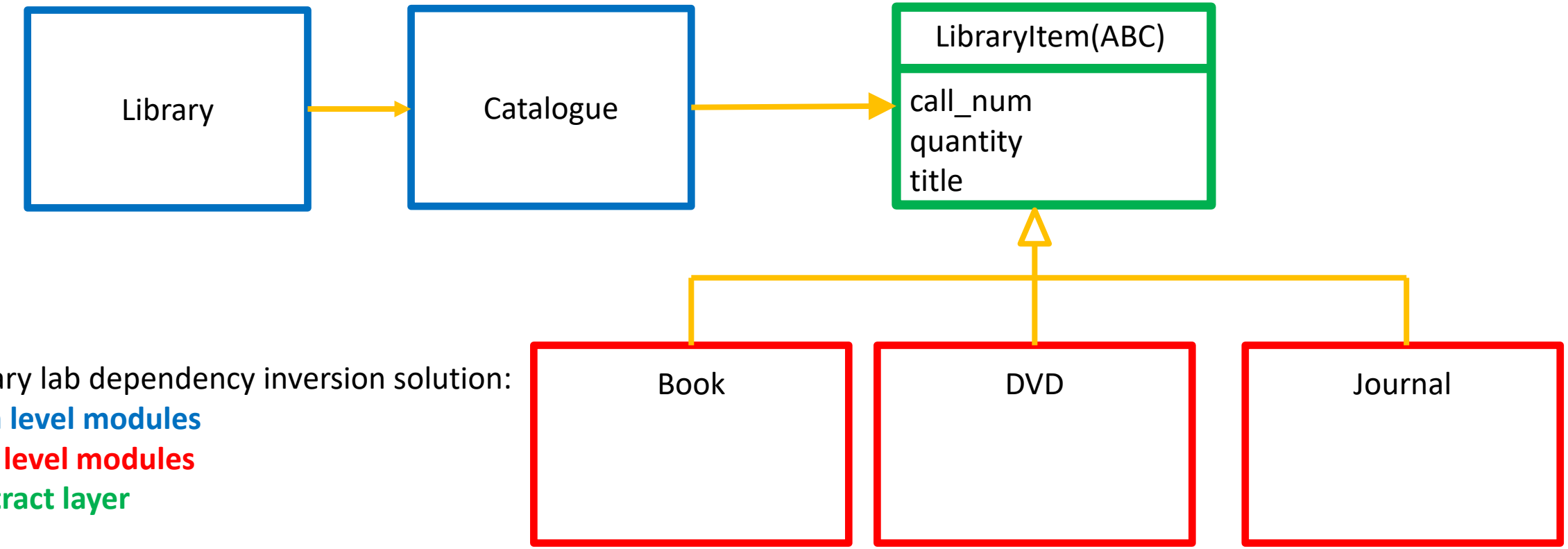
We introduced an **abstract layer** that does not depend on anything.

We made both our **high level** and **low level modules** **depend** on the **abstract** instead.

Now we can switch out either side, the UI and the text file could be replaced with something completely different



Dependency Inversion Principle



Library lab dependency inversion solution:

High level modules

Low level modules

Abstract layer

Law of Demeter

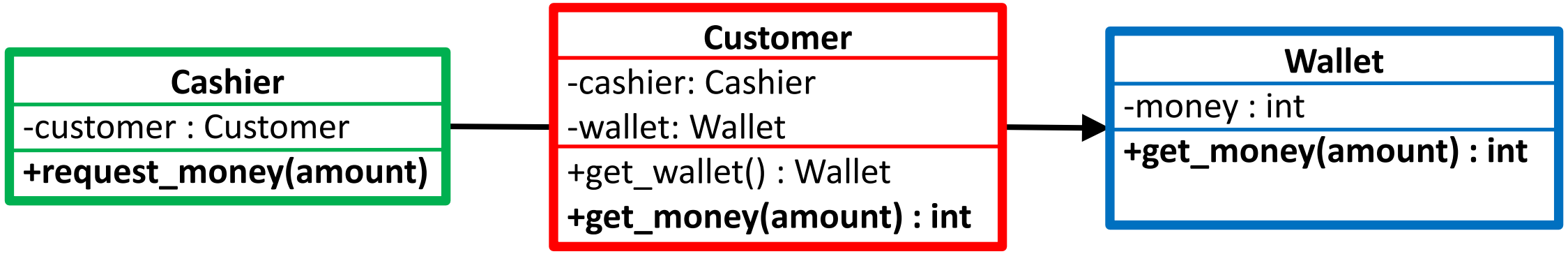
Real life - Wallet example

- Imagine if customer, cashier, wallet are classes
- Cashier should not reach into customer, then reach into their wallet to get the money
- Classes **should not** have knowledge of public members several classes away
- Cashier can access customer, but not wallet, through customer

Law of Demeter

- Imagine if **Cashier**, **Customer**, and **Wallet** are classes
- **Cashier** *should not* reach in **Customer**, then reach into their **Wallet** to get the money.
  - In **Cashier** **take_money()**: `money = self._customer.get_wallet().get_money(amount)` #BAD
- Classes **should not** have knowledge of public members several classes away

Law of Demeter



- **Cashier** can access **Customer**, but should not access **Wallet** through **Customer**
- Refactor code so **Cashier** requests money from **Customer**. **Customer** accesses own **Wallet** to give money
  - In **Cashier** **request_money()**: `money = self._customer.get_money(amount) #OK!`
  - In **Customer** **get_money()**: `return self._wallet.get_money(amount) #OK!`
- Classes **should** have knowledge of public members of **neighboring classes**

Law of Demeter

"Each unit should have only limited knowledge about other units: only units "closely" related to the current unit. **Each unit should only talk to its friends**; don't talk to strangers."

Don't have train wrecks like these:

```
obj.getX().getY().getZ().doSomething();
```

Or

```
MyCharacterUI.GetCharacterData().CreateCharacter().Save()
```

Poor Demeter is often forgotten, some people even call it "Suggestion of Demeter" since this is difficult to avoid

Law of Demeter – Example Code

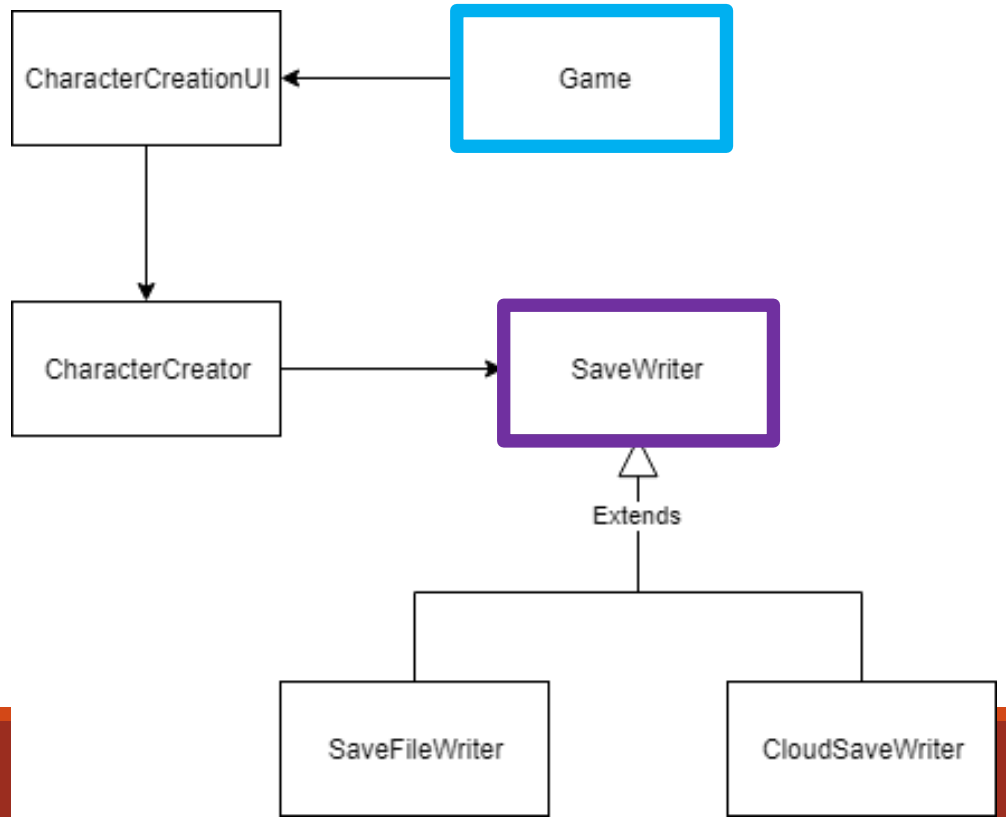
```
class Game:
    def create_character(self):
        self._data =
self.character_ui.get_character_creator().save(self.character_ui.get_character_creator().create_character())
```

This code tries to create a character and save it

This is obviously **bad** and **hard to read** code

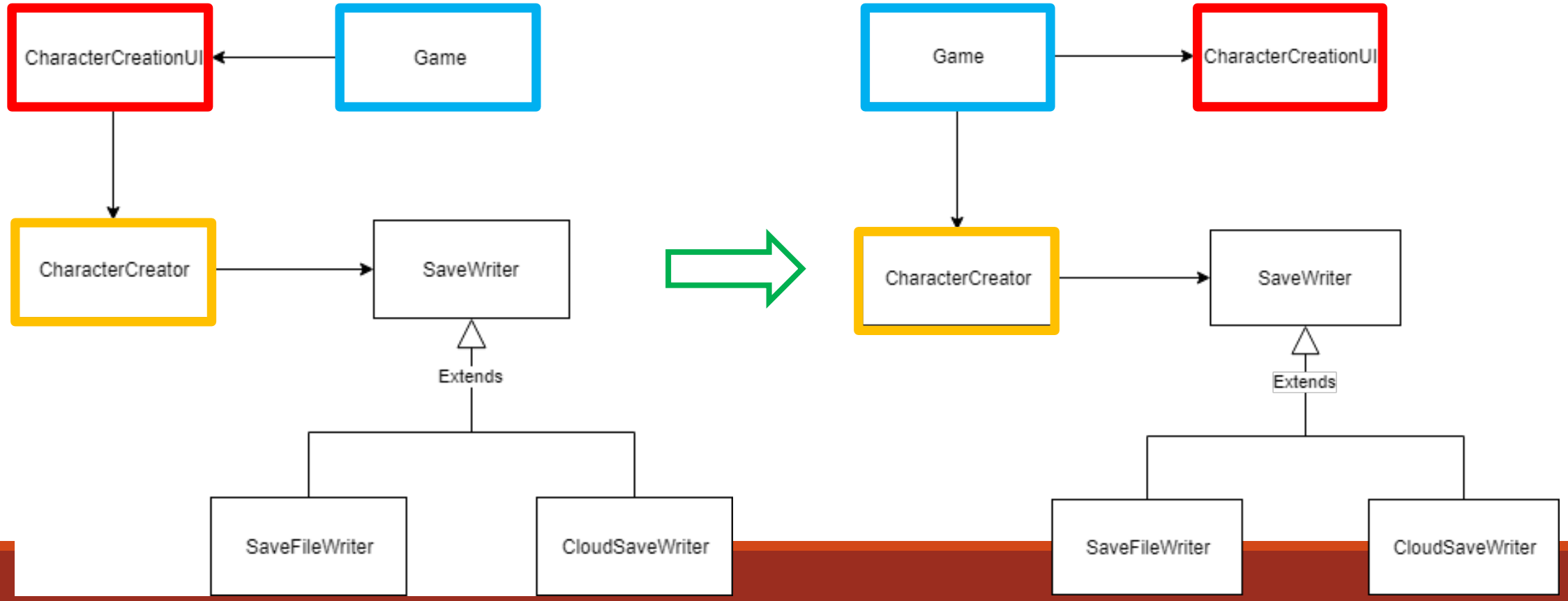
I also had a hard time getting the lines to fit in powerpoint. Bad for many reasons

Any change in the **save method**, which is all the way in **SaveWriter**, could have a ripple effect of changes all the way to our **Game** class



Law of Demeter – Example Code

- Part of the solution is to restructure the system by swapping **Game** and **CharacterCreationUI**
- Therefore **Game** does not need to *reach through* **CharacterCreationUI** to get **CharacterCreator**



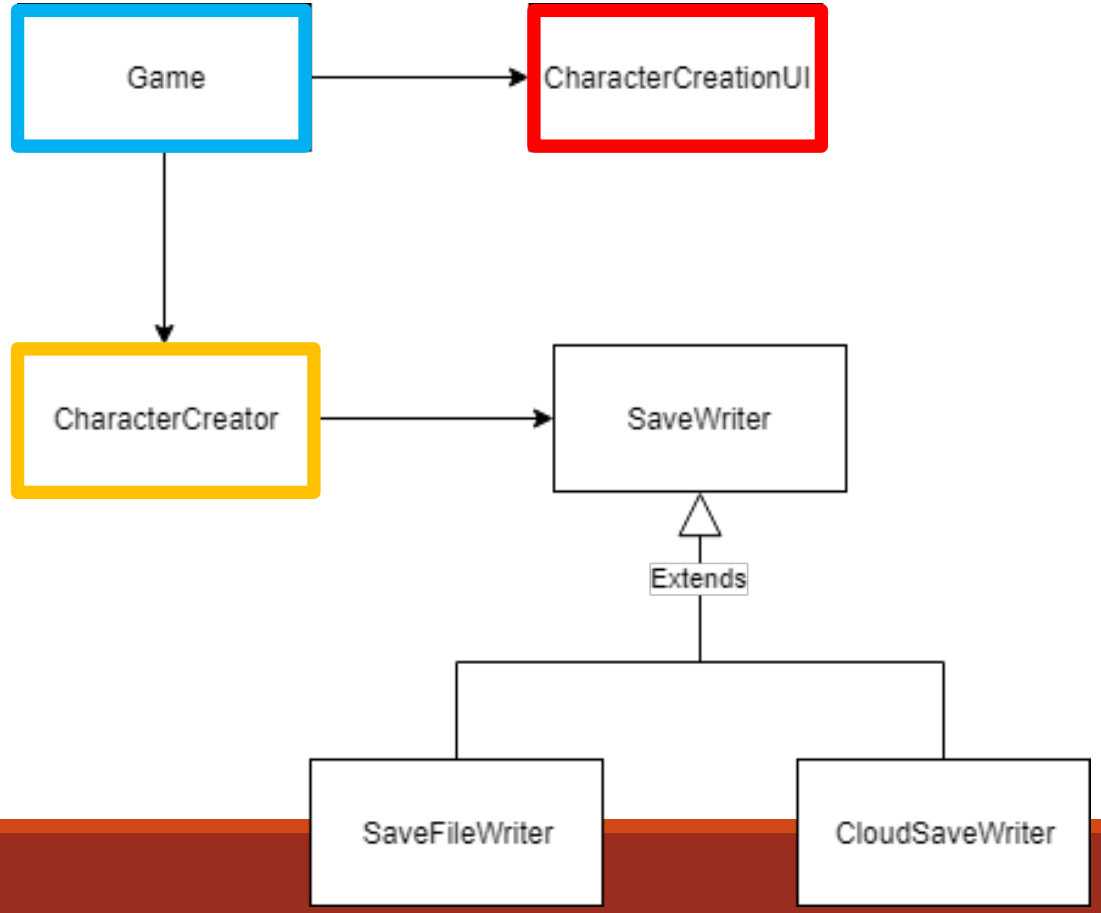
Law of Demeter – Example Code - Solution

Restructure code so each class is responsible for their own data. This prevents classes reaching deeply through multiple classes

```
class Game:
    def create_character(self):
        data = self.character_ui.get_data()
        self.character_creator.initiate_character_creation(data)

class CharacterCreationUI:
    def get_data(self):
        return self.character_data

class CharacterCreator:
    def initiate_character_creation(self, data):
        my_char = self.create_character(data)
        self.save_writer.save(my_char)
```



CLASS DIAGRAM DEPENDENCIES

Class diagram - Relationships

Arrows indicating how two classes are related

Dependency – A contains code that temporarily **use** B in some way. A **uses** B

A - - - -> B

```
class ClassA:
    def __init__(self):
        pass

    def a_method(self, object_b):
        object_b.b_method() # Dependency. Calls a
                                method in ClassB
        return ClassB() # Dependency. Instantiates ClassB
                          but doesn't save into self. attribute
```

- - - - - ->

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram - Relationships

Arrows indicating how two classes are related

Association – A contains data attributes B. A **has-a** B. General form of aggregation and composition

- Indicates **no conceptual** ownership, A does NOT own B.

- ie: Teacher → Student. Teacher teaches Students, but they don't conceptually own each other

A ⟶ B

```
class ClassA:
    def __init__(self, object_b):
        # Association
        # General term when ClassA saves ClassB as
            data attribute self._object_b or
            self._object_b_2
        self._object_b = ClassB()
        self._object_b_2 = object_b
```

→

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram - Relationships

Arrows indicating how two classes are related

Aggregation – A is aggregated with B. A **has-a** B, and if A is destroyed B still exists.

- Indicates **weak conceptual** ownership, A weakly owns B

- ie: Library ◇—— Book. Library owns Books. If Library closes, Books can still exist

A ◇—— B

```
class ClassA:
    def __init__(self, object_b):
        self._object_b = object_b # Aggregation. Specific
            form of association when ClassB passed in
            from external source AND ClassB saved as
            data attribute self._object_b
```

◇——

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram - Relationships

Arrows indicating how two classes are related

Composition – A is composed of B. A **has-a** B, and if A is destroyed, B also destroyed.

- Indicates **strong conceptual** ownership, A strongly owns B

- ie: House ◆—— Room. House owns Rooms. If House destroyed, so are the Rooms

A ◆—— B

```
class ClassA:
    def __init__(self):
        self._object_b = ClassB() # Composition. Specific
            form of association when ClassB
            instantiated in scope AND saved in data
            attribute self._object_b
```

◆——

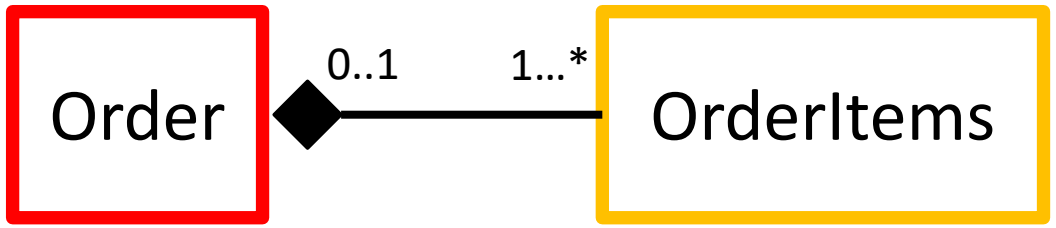
```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram

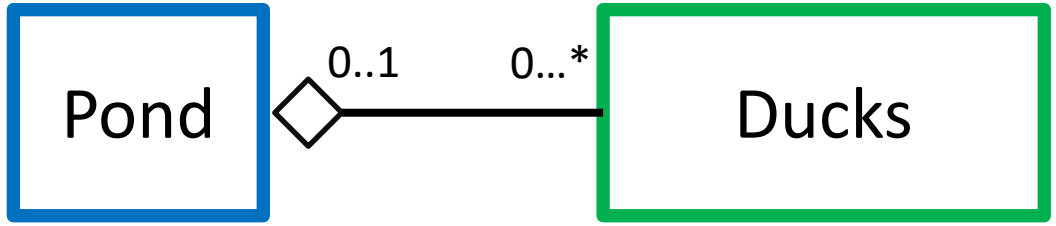
Composition: When a **class** is made up of and has references to **objects of another class**. In a composition relationship, if the **whole** is destroyed, the **parts** cannot exist.

Eg: If an ***Order*** is destroyed, then so are the ***OrderItems*** of which it is composed.



Aggregation: When a **class** references an **object of another class**. In an aggregation relationship, if the **class** referencing it is destroyed, the **referenced class** can live on.

Eg: If a ***Pond*** is destroyed, the **Ducks** can live on.



Class diagram - Relationships

Arrows indicating how two classes are related

Inheritance/Generalization – A is a subclass of B. A **is-a** B for concrete or abstract classes

A ⟶ B

```
class ClassA(ClassB): # Inheritance, ClassA inherits from ClassB
    pass
```

⟶

```
class ClassB:
    def __init__(self):
        pass

    def b_method(self):
        pass
```

Class diagram - Relationships

Arrows indicating how two classes are related

Realization/Implementation – A is implementing interface B. A **is-a** B for interfaces

A - - ▷ B

```
class ClassA(ClassB): # Realization. ClassA inherits from ClassB and implements ClassB abstract methods
    def function(self):
        pass
```

- - ▷

```
class ClassB(abc.ABC):

    #ClassB ONLY has abstract methods, no data attributes or implemented methods
    @abc.abstractmethod
    def function(self):
        pass
```

uml_review.py

POSITIONAL ARGUMENTS, ARGS, KWARGS

Positional arguments

```
def my_func(a_list, a_num, a_string):
    print(a_list, a_num, a_string)
```

```
my_func(['cat','dog'], 5, 'hello')
```

The order that parameters passed in matters with **positional arguments** (this is the one we're used to). Errors will occur otherwise

Named Arguments: Changing the Order

```
def my_func(a_list, a_num, a_string):
    print(a_list, a_num, a_string)
```

```
my_func(a_num=5, a_string='hello', a_list=['cat','dog'])
```

The **position of parameters that are passed in doesn't matter with named arguments**. All that matters is explicitly assigning a value to a matching parameter name

Named Arguments: Changing the Order

```
def my_func(a_list, a_num=0, a_string=''):
    print(a_list, a_num, a_string)


my_func(a_num=5, a_list=['cat','dog'])
```

This is especially useful when functions have **default parameters**. Don't need to pass in all parameters. Only need to pass in parameters we care about

Variable List Arguments

What if we wanted to provide a variable number of parameters?

E.g:

```
my_print("COMP3522", "is great.", "Tuesdays are long")
my_print("I love", "weekends!")
```

We could have the function expect a list and iterate through it.

Or maybe there is a better way to do this since Python is cleaner and nicer?

Variable Argument List - “Packing Arguments”

We specify an argument with an * preceding its name. The standard/convention is ***args**

This is known as **packing** arguments. All arguments are packed into a **tuple** named **args**

```
def my_print(*args):
    for item in args:
        print(item)

my_print('dog', 5, {1:'hi', 2:'hello'})
```

Variable Argument List - “Unpacking”

The * (asterisk) can be used with **containers** as well. This is known as **unpacking**

In the example below, the **list** is “**unpacked**” into the individual string, integer, and list objects

```
def my_print(in_str, in_int, in_dict):
    print(in_str, in_int, in_dict)

my_list = ['dog', 5, {1: 'hi', 2: 'hello'}]
my_print(*my_list) #unpack list
```

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

Looks like we're passing in named arguments

Also notice how this looks like a dictionary without the **{}**

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

There are keys

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

There are keys, and **values**

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

****kwargs** interprets the key/value parameters passed in as a dictionary

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

kwargs is a dictionary, so in order to get the key/value pairs, we need to explicitly call the **items method**

Notice how we're using *unpacking* to get the key/value pairs from the dictionary

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

- Each key/value pair can be extracted using **key**, **value** variables
- Note that key and value are NOT special names, they could be replaced by x, y and the code behaves the same

positional_named_args_kwargs_review.py